

Position Embedding / Positional Encoding

深度分析：从 Sinusoidal、Relative Bias 到 RoPE 与百万级长上下文

Algorithms · Gotchas · Know-how · Know-why · Long-context Engineering

核心问题

Transformer 的 Attention 本身对顺序没有天然感知。Position Encoding 不是一个附属 feature，而是模型的“地址系统”：它决定 token 如何区分先后、如何表达相对距离、如何外推到训练长度之外，以及长上下文时 attention 的数值与几何结构。

研究版 · 2026-08

基于原始论文、技术报告与主流开源实现核验

执行摘要：先记住这 12 个结论

1. Self-Attention 在没有位置机制时是 permutation-equivariant：交换输入 token 的位置，会相应交换输出，而不会自动理解“第几个”。
2. “Position Embedding”并非单一算法。至少应区分：绝对 additive embedding、pairwise relative representation、attention-logit bias、Q/K 几何变换（RoPE）四类。
3. 原始 Sinusoidal Encoding 的本质不是“给每个位置一个随机向量”，而是用多尺度正弦波构造可解析的连续坐标系；不同维度对应不同波长。
4. Learned Absolute Embedding 简单有效，但最大位置往往被参数表大小和训练分布锁定；直接访问未训练位置没有可靠语义。
5. Relative Position 的关键优势是把“地址”从绝对索引 i 转向 pairwise offset $i-j$ ，更贴近注意力真正需要的关系。
6. T5 Relative Bias 极其简洁：将相对距离分桶后，只向 attention logits 加一个可学习标量；远距离采用对数分桶以节省参数。
7. ALiBi 不学习位置向量，而是按距离给 attention score 加线性惩罚，内置 recency bias，因此天然具备一定长度外推倾向。
8. RoPE 的关键不是“旋转看起来优雅”，而是 $q_m^T k_n$ 可化成只依赖 $m-n$ 的相对旋转，同时保持 Q/K 向量范数。
9. RoPE 的频率谱是长上下文的核心。高频维度负责局部位置分辨率，低频维度负责大范围 field；context extension 本质上是在重分配这组频率/相位。
10. 仅把 max_position_embeddings 改大通常不等于长上下文能力。RoPE 在训练窗口之外会进入 OOD 相位区域；需要 PI、YaRN、LongRoPE、LongRoPE2 或模型原生的 scaling recipe，并通常配合长上下文训练。
11. “模型能接受 128K tokens”与“模型能有效使用 128K tokens”是两回事。Lost-in-the-Middle 与 RULER 都表明，标称窗口经常显著大于有效窗口。
12. 位置机制最终必须和训练长度分布、GQA/KV Cache、FlashAttention、sliding-window attention、混合精度、长上下文数据和 benchmark 一起设计，而不是只看公式。



Evolution of positional representation: absolute address → relative relation → geometric scaling

图 1 *Position representation* 的主要演化路线（概念化时间轴）

目录

1. 为什么 Transformer 必须显式注入位置
2. Position Embedding 的术语与算法分类
3. Sinusoidal Positional Encoding: 从公式到频率直觉
4. Learned Absolute Position Embedding
5. Shaw Relative Position Representation
6. Transformer-XL: 相对位置与跨 segment recurrence
7. T5 Relative Position Bias
8. ALiBi: 用线性距离偏置换取外推
9. RoPE: Rotary Position Embedding 的完整推导
10. RoPE 的频率、波长、base theta 与 “位置分辨率”
11. RoPE 工程实现: shape、partial rotary、GQA 与 KV cache
12. 为什么长上下文会失败: extrapolation、OOD phase 与 attention geometry
13. Context Extension 方法族: PI、Dynamic/NTK、YaRN、LongRoPE、LongRoPE2
14. xPos、CoPE 与其他方向
15. 有效上下文 $\neq$ 标称上下文: 评测方法
16. 数值稳定性: BF16/FP16、cos/sin 与长序列
17. 2D / 多模态位置: Vision、Image、Video、M-RoPE
18. Position Encoding 与系统设计: FlashAttention、Sliding Window、Cache
19. Gotchas: 35+ 个高频坑
20. Know-how: 训练、扩窗与迁移操作手册
21. Know-why: 核心设计背后的因果逻辑
22. Ablation 与评测框架
23. 方法选择决策树
24. PyTorch From Scratch 代码
25. 深度面试题 / Debug 题
26. 参考文献

1. 为什么 Transformer 必须显式注入位置

Transformer 的 Self-Attention 先计算 $Q = XW_Q$ 、 $K = XW_K$ 、 $V = XW_V$ ，再通过 QK^T 对所有 token 建立关系。若输入只包含 token embedding，而没有任何位置相关的信号，那么这一计算对 token 的排列具有对称性：对输入做同一个 permutation，输出会做相同 permutation。模型知道“有哪些 token”，却没有一个天然的坐标系告诉它“谁在前、谁在后、相隔多远”。

$$\text{Attention}(X) = \text{softmax}(QK^S / \sqrt{d_k}) V$$

因此，位置机制本质上是在破坏这种排列对称性。它可以作用在四个不同的位置：输入表示、Q/K/V、attention logits、或 attention pattern/结构本身。不同方案的外推能力、参数量和系统代价，主要由“位置被注入到哪里”决定。

Know-why

语言顺序不是一个普通 feature。它决定语法角色、指代、因果顺序、代码执行顺序、数学推导顺序。一个没有地址系统的 Transformer，可以强大地建模集合关系，却不能仅凭 attention 算子本身区分“dog bites man”与“man bites dog”。

2. Position Embedding 的术语与算法分类

类别	注入位置	代表方法	核心变量	外推倾向
Absolute additive	X 输入/第一层 hidden	Sinusoidal, Learned Absolute	position i	固定函数可计算新位置；learned table 通常弱
Relative vector	Q/K/V 或 score 的 pairwise 项	Shaw, Transformer-XL	offset i-j	通常比绝对索引自然
Attention bias	attention logits	T5 bias, ALiBi	bucket(i-j) / distance	低参数、易与 attention kernel 融合
Geometric transform	Q/K	RoPE, xPos	phase/frequency × position	现代 LLM 主流；扩窗需处理频率 OOD
Context-dependent address	attention / learned counter	CoPE	内容决定“位置增量”	研究方向：位置不必等于 token count

“Encoding”常用于固定函数（例如 sinusoidal），“Embedding”常用于可学习 lookup table，但论文和代码实践并不严格区分这两个词。工程讨论时，更重要的是确认：是否有参数、是否依赖绝对索引、是否依赖 pairwise distance、位置是否进入 attention logits，以及是否改变 Q/K 的几何。

3. Sinusoidal Positional Encoding：从公式到频率直觉

原始 Transformer 使用不同频率的 sin/cos 作为位置函数，并将其直接加到 token embedding。对偶数/奇数维度：

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

可把每一对维度看成一个二维单位圆上的相位。低维使用更高频率，对位置变化敏感；高维使用更低频率，提供更长范围的坐标变化。这样同一位置同时拥有“局部秒针”和“长期时针”式多尺度表示。

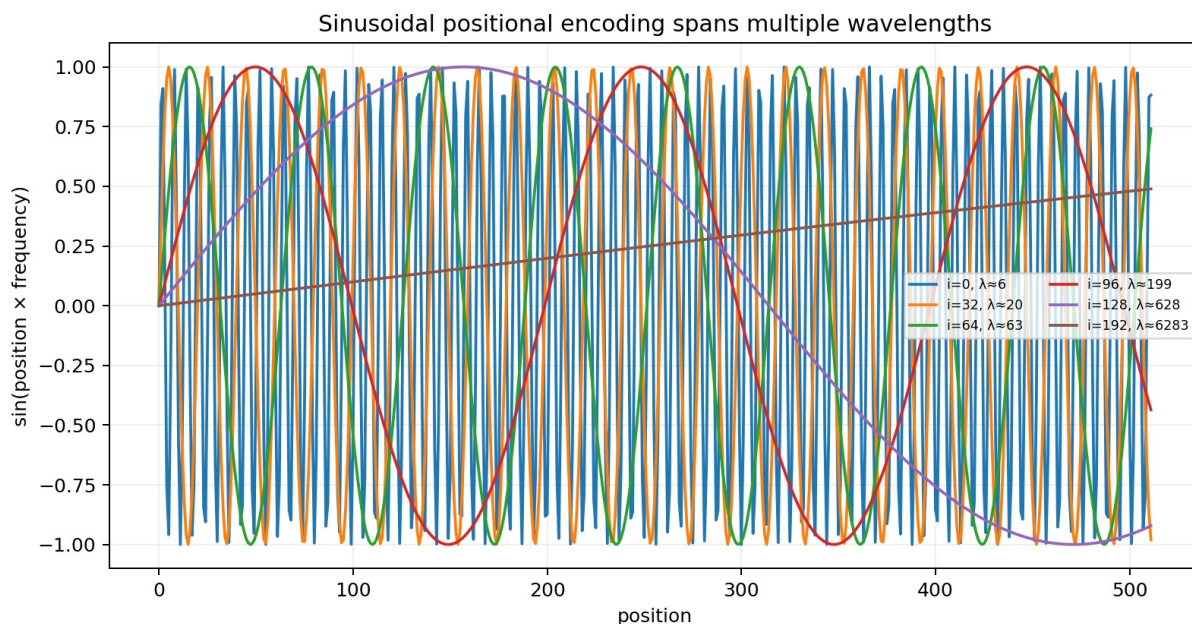


图2 Sinusoidal PE 的不同维度对应不同波长；高频更容易区分邻近位置，低频覆盖更长距离。

优点	原因
无位置表参数	position → vector 由固定函数直接计算
理论上可生成任意 position	不像 learned table 必须预先分配 max_position
相对位移可由线性关系表达	$\sin(a+b)$ 、 $\cos(a+b)$ 可由 $\sin/\cos(a)$ 的线性组合得到
多尺度	不同 wavelength 同时覆盖近距离与远距离

Gotcha：可计算 ≠ 可外推

Sinusoidal 函数可以在训练长度之外计算数值，但模型权重未必学习过这些相位组合。因此“数学上可生成任意位置向量”不能自动推出“模型对任意长度泛化”。

4. Learned Absolute Position Embedding

Learned Absolute Position Embedding 建立一个 $P \in \mathbb{R}^{(L_{\max} \times d)}$ 的位置表，第 i 行代表第 i 个位置，再与 token embedding 相加。BERT、GPT-2 等经典模型使用过类似形式。它把“位置编码设计”交给梯度学习，通常在训练长度内很强，但位置表本身是有限参数集合。

$$\mathbf{h}_i^0 = \text{token_embed}(\mathbf{x}_i) + \text{position_table}[i]$$

维度	Learned Absolute	Sinusoidal
参数量	$L_{\max} \times d$	0
训练内适配	强，可针对任务优化	固定
未训练新位置	没有已学习 row；通常需要插值/扩表	函数可计算
长度迁移	通常需要 position interpolation / resize	仍需验证模型是否真正外推
实现	最简单 lookup	$\sin/\cos$ 预计算或在线生成

Gotcha

直接把 position table 从 2K resize 到 8K，仅解决 tensor shape，不保证新增位置具有正确语义。视觉 Transformer 常见 2D interpolation 在图像分辨率变化时可工作，但 NLP 中长度分布、因果 attention 与 token distance 的含义不同，不能机械照搬。

5. Shaw Relative Position Representation

Shaw et al. 2018 将自注意力看成一个全连接有向图，并为边 (i, j) 引入表示其相对距离的向量。关键转变是：模型不再只问“token j 的绝对索引是多少”，而是问“ j 相对于 query i 在哪里”。

$$\mathbf{e}_{ij} = \mathbf{q}_i^s (\mathbf{k}_j + \mathbf{a}^{\mathbf{K}_{\{ij\}}}) / \sqrt{d_k}$$
$$\mathbf{z}_i = \sum_j \alpha_{ij} (\mathbf{v}_j + \mathbf{a}^{\mathbf{V}_{\{ij\}}})$$

通常相对距离会 clipping 到一个有限窗口，例如 $\text{clip}(i-j, -k, k)$ ，这样参数量与最大序列长度解耦。论文在机器翻译上显示，相对位置可以替代绝对位置并取得改进。

Know-why

Attention 的核心是 pairwise interaction。把位置定义为 $i-j$ ，直接把位置信息放到了“关系”上，而不是先把一个绝对位置向量混到 token 内容中再等待网络自己解耦。

6. Transformer-XL：相对位置与跨 Segment Recurrence

Transformer-XL 的难题来自 segment recurrence：缓存上一段 hidden state 后，如果仍使用绝对位置，旧 segment 和新 segment 的绝对坐标语义会发生冲突。Transformer-XL 因此重新参数化 attention score，使内容-内容、内容-位置、全局内容偏置、全局位置偏置分解，并使用相对距离编码。

$$\mathbf{A}^{\text{rel}_{\{i,j\}}} = \mathbf{E}_{\{\mathbf{x}_i\}}^s \mathbf{W}_{\mathbf{q}}^s \mathbf{W}_{\{\mathbf{k}, \mathbf{E}\}} \mathbf{E}_{\{\mathbf{x}_j\}} + \mathbf{E}_{\{\mathbf{x}_i\}}^s \mathbf{W}_{\mathbf{q}}^s \mathbf{W}_{\{\mathbf{k}, \mathbf{R}\}} \mathbf{R}_{\{i-j\}} + \mathbf{u}^s \mathbf{W}_{\{\mathbf{k}, \mathbf{E}\}} \mathbf{E}_{\{\mathbf{x}_j\}} + \mathbf{v}^s \mathbf{W}_{\{\mathbf{k}, \mathbf{R}\}} \mathbf{R}_{\{i-j\}}$$

这种设计让同一 query 在跨 segment memory 中仍能按“相隔多少”理解 key，而不是依赖已经过期的绝对 row。

Know-how

当模型引入 recurrence、memory token、跨 chunk KV reuse 时，优先检查位置机制是否保持“跨 chunk 的相对坐标一致性”。很多长上下文 bug 本质上不是 attention kernel 错，而是 position_ids 在 chunk 边界被重置或重复。

7. T5 Relative Position Bias

T5 把相对位置设计进一步简化：不再给相对位置一个 d 维向量，而是给 attention logit 加一个标量 bias。不同 attention head 有自己的 bias；参数在各层共享。T5 使用 32 个 relative-position buckets，并让 bucket 的距离范围随距离增加而近似对数增长；论文设置到 offset 128，超过后映射到同一最远 bucket。[4]

$$\text{score}_{ij} = \mathbf{q}_i^s \mathbf{k}_j / \sqrt{d_k} + \mathbf{b}_{\text{head}}(\text{bucket}(i-j))$$

设计	Why
标量而非向量	位置主要用于影响“看哪里”，无需额外改变 value 内容
分桶	不用为每一个距离单独分配参数

近距离细分、远距离粗分	语言对局部词序通常更敏感；远距离只需粗粒度
跨层共享	降低参数量，并使位置先验一致

Gotcha

T5 的最远 bucket 并不表示“模型只能看 128 tokens”。它表示单层的位置 bias 对 >128 的距离不再区分精细值；深层仍可通过内容传播和多层组合形成更大范围依赖。

8. ALiBi : Attention with Linear Biases

ALiBi 完全取消位置向量，在 causal attention logit 上按 query-key 距离加一个负的线性 bias：距离越远，惩罚越大；不同 head 使用不同 slope。

$$\text{score_h}(\mathbf{i}, \mathbf{j}) = \mathbf{q_i}^s \mathbf{k_j} / \sqrt{d_k} - \mathbf{m_h} \cdot (\mathbf{i} - \mathbf{j}), \quad \mathbf{j} \leq \mathbf{i}$$

论文展示了“train short, test long”的外推能力：在 1024 长度训练的 1.3B 模型上外推到 2048，并报告相对更长训练 baseline 的速度/内存优势。[7]

优点	代价/偏置
零新增 position embedding 参数	把 recency 偏好写死进 attention prior
实现简单	head slope 设计影响远近尺度
不需要 sin/cos	超远距离 bias 绝对值不断增大
一定长度外推能力	外推不是无限保证；数据/训练仍重要

Know-why

ALiBi 将“越近通常越相关”的语言统计直接作为 inductive bias。它不像 RoPE 那样用相位表达相对位置，而是用 attention energy 的线性斜坡表达距离成本。

9. RoPE : Rotary Position Embedding 的完整推导

RoPE 是现代 decoder LLM 最常见的位置机制之一。它不把位置向量加到 X，而是在计算 attention 前，对每个 head 的 Q 和 K 以 position-dependent 的角度做二维旋转。

$$\mathbf{R}(\mathbf{m}, \theta) = [[\cos(\mathbf{m}\theta), -\sin(\mathbf{m}\theta)], [\sin(\mathbf{m}\theta), \cos(\mathbf{m}\theta)]]$$

$$\mathbf{q_m} = \mathbf{R}(\mathbf{m}, \theta) \mathbf{q}, \quad \mathbf{k_n} = \mathbf{R}(\mathbf{n}, \theta) \mathbf{k}$$

由于旋转矩阵是正交矩阵， $\mathbf{R}(\mathbf{m})^T \mathbf{R}(\mathbf{n}) = \mathbf{R}(\mathbf{n} - \mathbf{m})$ ，于是：

$$\mathbf{q_m}^s \mathbf{k_n} = \mathbf{q}^s \mathbf{R}(\mathbf{n} - \mathbf{m}, \theta) \mathbf{k}$$

这一步是 RoPE 的核心：每个 token 的绝对位置通过旋转相位编码，但最终 Q·K 的位置效应只依赖相对位移 n-m。与此同时，旋转保持向量范数不变，不会像 additive embedding 那样直接改变 Q/K magnitude。

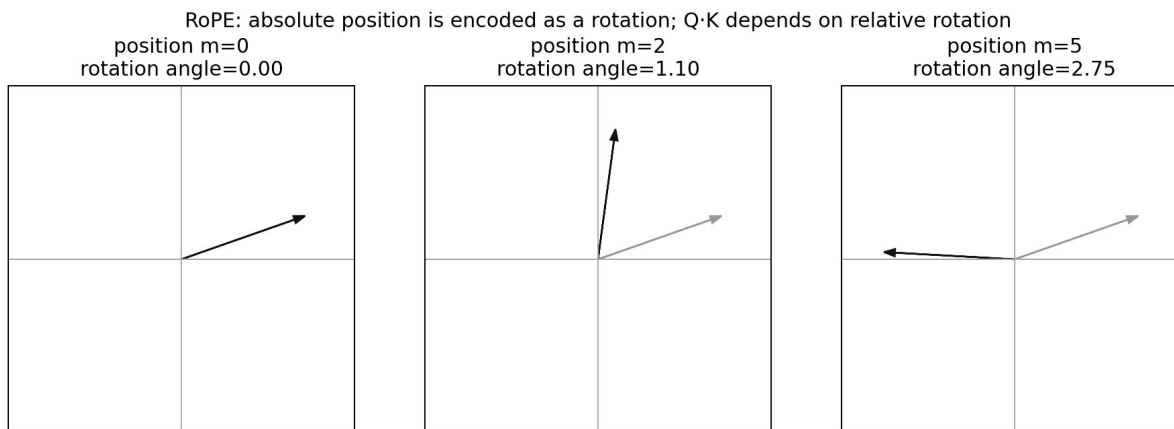


图3 RoPE 的二维几何：同一内容向量在不同 position 被不同角度旋转；两向量点积包含相对旋转。

9.1 复数视角

把每两个维度看成复数 $z = x_0 + i x_1$ ，则 RoPE 等价于乘上 $e^{(i m \theta)}$ 。因此 query-key 内积可以理解为不同频率复相位的叠加。这个视角非常适合分析长上下文：当 m 很大时，不同频率分量经历不同数量的旋转周期，训练外位置可能产生从未见过的相位组合。

$$\text{RoPE}(z, m, \theta) = z \cdot \exp(i m \theta)$$

10. RoPE 的频率、波长、base θ 与位置分辨率

标准 RoPE 通常为第 i 对维度设置 $\theta_i = \text{base}^{(-2i/d_{\text{head}})}$ ，base 常见为 10000，但现代模型可能使用更大的 rope_theta。每个 θ_i 对应波长 $\lambda_i = 2\pi/\theta_i$ 。

$$\theta_i = \text{base}^{(-2i/d)}, \quad \lambda_i = 2\pi / \theta_i$$

频率区域	作用直觉	长上下文风险
高频 / 短波长	高位置分辨率，区分相邻或近距离 token	长距离下快速循环，多次 phase wrap
中频	平衡局部与中距离关系	扩窗时可能进入训练未覆盖角度组合
低频 / 长波长	覆盖大 field，远距离变化慢	训练窗口内可能只经历不完整旋转，导致某些维度“未充分训练”

2025-2026 的研究进一步把 RoPE 看成“频率资源分配问题”：不同 head/维度对频率的利用并不均匀；长距离检索时某些高旋转维度可能低效，而训练数据的依赖尺度会影响模型最终依赖哪些频率。[18][24] 这些属于活跃研究结论，应与 RoPE 的基本数学事实区分。

Gotcha: rope_theta 不是“上下文长度参数”

增大 rope_theta 会降低一部分旋转频率、延长波长，因此可能有利于更长距离，但它改变了模型训练时的位置几何。对已有 checkpoint 直接改 theta，等于让同一个 position_id 产生不同 Q/K 旋转，通常需要配套验证或继续训练。

11. RoPE 工程实现：Shape、Partial Rotary、GQA 与 KV Cache

典型 decoder attention 中，Q/K reshape 为 [batch, heads, seq, head_dim]。RoPE 通常只操作最后的 head_dim，并按偶/奇维配对。GQA 中 Q head 数量大于 KV head 数量，但 K 的旋转必须在 KV head 自己的维度上按同一 position_id 进行，再进行 repeat/interleave 或 kernel 内 broadcasting。

```
# Conceptual pseudo-code
q = q_proj(x).view(B, T, n_q_heads, head_dim).transpose(1, 2)
k = k_proj(x).view(B, T, n_kv_heads, head_dim).transpose(1, 2)

cos, sin = rope(position_ids)          # [B or 1, T, rotary_dim]
q = apply_rotary(q, cos, sin, rotary_dim)
k = apply_rotary(k, cos, sin, rotary_dim)

# Cache *rotated* K with the position used at insertion time.
kv_cache.append(k, v, position_ids)
```

Gotcha	后果
Q/K position_ids 不一致	相对位移被错误编码；attention 可能还能跑但语义错
cache 中 K 重复旋转	相位叠加，生成质量迅速异常
prefill/decode position offset 错	短 prompt 正常，长对话或 batch decode 出错
padding 左/右策略与 position_ids 不一致	同一文本因 padding 产生不同位置几何
partial_rotary_factor 改错	checkpoint 权重假设被破坏；未旋转维度比例不一致
GQA repeat 前后次序错误	不同 Q heads 可能对应错误的 K 相位

Hugging Face 当前 RoPE 配置支持 default、linear、dynamic、yarn、longrope、llama3 等类型，并允许 partial_rotary_factor，以及按 layer type 设置不同 RoPE 参数。[15] 这说明工程上 RoPE 已经从“一个公式”演化成模型配置中的一套策略族。

12. 为什么长上下文会失败：Extrapolation、OOD Phase 与 Attention Geometry

假设模型预训练只看到 0…4095。直接把 position_id 用到 30000，虽然 sin/cos 有数值，但 RoPE 频率组合可能进入从未被训练数据覆盖的相位区域。Position Interpolation 论文指出，直接 extrapolation 可能产生灾难性的 attention score；其策略是把新位置压缩回原位置范围，而不是让角度直接跑出训练域。[10]

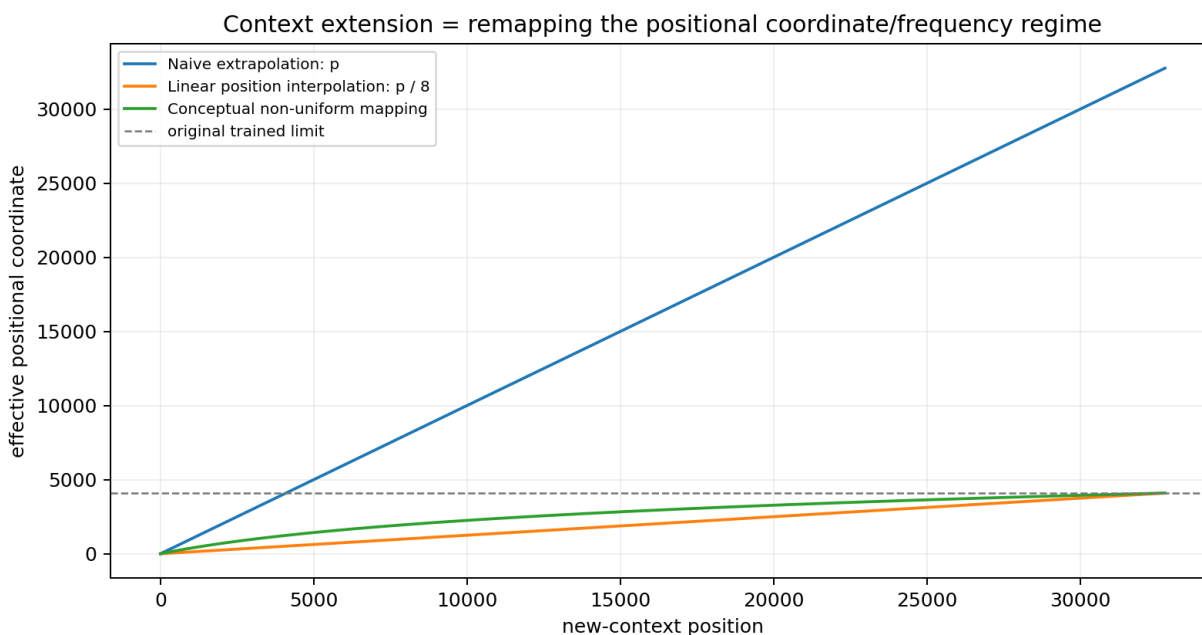


图 4 长上下文扩展的核心视角：不是只把数组长度加大，而是重新映射 position/frequency，使新上下文尽量落入可学习的相位区域。

12.1 三种“长度”的区别

长度概念	定义	常见误区
Architectural max	实现允许的 position/cache/attention shape 上限	能 allocate 不代表模型会用
Training max / distribution	训练时真正出现的 sequence length 与 distance 分布	偶尔出现少量长样本 $\neq$ 学会长距离推理
Effective context length	任务性能仍可靠的最大范围	通常小于 marketing/config 的 max

13. Context Extension 方法族

13.1 Linear Position Interpolation (PI)

若原窗口 L ，目标窗口 $L' = sL$ ，则用位置 p/s 代替 p 。这样新最大 position 被压回原训练范围。优点是简单稳定，缺点是所有距离都被统一压缩，局部位置分辨率降低。PI 在 LLaMA 上展示了以少量 fine-tuning 将窗口扩到 32K。[10]

$$p_{\text{eff}} = p / s, \quad s = L_{\text{target}} / L_{\text{train}}$$

13.2 Dynamic / NTK-aware scaling (工程族)

社区与主流库中存在通过改变 RoPE base/frequency 而非只缩放 position 的动态 scaling 方案，目标是在局部高频分辨率与远距离低频覆盖之间取得更好平衡。该类方法有多种实现/命名，checkpoint 的 config 必须视为算法的一部分；不要只根据“NTK”标签假设不同库行为一致。

13.3 YaRN

YaRN 将 RoPE extension 视为频率分区和 attention magnitude 校准问题：对不同频率使用不同的 interpolation/extrapolation 混合，并引入 attention scaling。论文报告相对此前方法减少训练 token 与步骤，并在 LLaMA 系列上扩展到更长上下文。[11]

13.4 LongRoPE

LongRoPE 进一步利用非均匀性：不同 RoPE 维度不必使用相同缩放系数；不同 position region 也不必同等处理。其通过搜索找到每维 scaling，并使用 progressive extension 先扩到 256K，再做第二阶段插值到 2M 级，同时做短上下文 readjustment 以恢复原始性能。[12]

13.5 LongRoPE2

LongRoPE2 提出高维/低频 RoPE 在原始窗口中可能“训练不充分”，并用 needle-driven perplexity 引导 evolutionary search 选择 rescaling，再通过 mixed-context training 同时保留短上下文与长上下文性能。论文在 Llama-3-8B / Phi-3 mini 上报告了有效 128K 扩展，并强调“effective context”而非仅能输入的长度。[13]

13.6 Llama 3 / Model-specific RoPE scaling

现代模型常把 RoPE extension recipe 固化在 config 中。Hugging Face 的 rope utility 已把 llama3 作为独立 rope_type，与 linear、dynamic、yarn、longrope 区分。[15] 生产推理时必须读取 checkpoint 自带 config，而不是用框架默认 rope_theta/scaling 猜测。

方法	主要操作对象	需要训练?	主要 trade-off
Naive extrapolation	position 原样增大	不一定	简单但 OOD 风险最高
PI	统一缩小 position	通常少量	稳定；局部分辨率压缩
Dynamic/NTK family	base / frequencies 随扩展改变	视实现	兼顾频率，但实现差异大
YaRN	频率分区 + interpolation/extrapolation + attention scaling	少量	较强且工程成熟
LongRoPE	每维/位置非均匀 scaling + progressive	少量/搜索	效果强，搜索和配置复杂
LongRoPE2	search + mixed-window training	需要	强调近无损短/长兼容

14. xPos、CoPE 与其他方向

14.1 xPos

xPos 在 RoPE-like 旋转基础上加入与 position 相关的缩放，希望提升 attention resolution 与 length extrapolation。它代表一个重要方向：旋转只控制 phase，而 amplitude/scale 也可以携带有利于外推的结构。[9]

14.2 Contextual Position Encoding (CoPE)

CoPE 质疑“position 必须等于 token count”这一默认假设。它让 position increment 由内容决定，从而可以学习“第几个重要 token / 第几句 / 第几个满足某条件的元素”。论文在 selective copy、counting、Flip-Flop 以及语言/代码建模上报告收益。[19]

绝对 token index 是一种人为坐标系。对于代码块、句子、事件、视觉 patch 等结构，“语义位置”可能比“第几个 token”更有意义。CoPE 的价值在于把 position 从静态元数据变成可学习的 addressing 过程。

14.3 新研究应如何看

2025-2026 出现大量 RoPE 频率、head-specific scaling、hyperbolic/complex PE、position discrimination 等工作。报告将它们作为“研究趋势”而不是默认最佳实践：如果没有多模型、多长度和训练后复现实验，不应仅凭单论文把生产系统迁移到新位置编码。

15. 有效上下文 ≠ 标称上下文：评测方法

Lost in the Middle 发现，相关证据位于上下文中间时性能常显著下降；模型通常更擅长利用开头或结尾信息。[16] RULER 进一步指出，简单 needle-in-a-haystack 只测试浅层 retrieval，加入 multiple needles、multi-hop tracing、aggregation 后，很多标称 ≥32K 的模型在长度增加时性能明显下降。[17]

评测层级	建议任务	测什么
LM sanity	held-out perplexity vs length	位置扩展是否破坏基本语言建模
Single retrieval	passkey / NIAH	能否从长 context 找一个 key
Position robustness	needle 放在 0/25/50/75/100%	是否 Lost-in-the-Middle
Multi-needle	多个 key / distractors	容量与干扰
Multi-hop	跨段链式引用	不只检索，还需组合
Aggregation	count/sum/group over long context	全局覆盖
Real tasks	long QA, code repo, legal docs	迁移到真实分布
Short-context retention	原 benchmark / short PPL	扩窗是否损害原能力

Gotcha

只做 passkey retrieval 很容易高估长上下文能力。一个模型可以通过强模式匹配找到唯一 needle，却在多跳、聚合、位置鲁棒性或真实长文推理上失败。

16. 数值稳定性：BF16/FP16、cos/sin 与长序列

RoPE 的数学性质假设位置角度和旋转计算足够精确。2024 的研究报告 BF16 在长上下文下会让 RoPE 偏离理想相对位置性质，且误差随 context 增长积累。[20] 这提醒工程系统：位置编码不仅是“数学公式正确”，还必须检查实际 dtype 和 kernel。

实践	原因
position_ids 使用 int64/int32，不用低精浮点累计	大整数在 BF16/FP16 中不能逐个精确表示
cos/sin phase 构造优先 FP32	避免大 position × inv_freq 的量化误差
最终 cos/sin 可按 kernel 需要 cast	把关键相位计算和 tensor compute 分开
长上下文训练做 dtype ablation	短上下文正常并不能证明 128K 稳定
检查 chunked prefill 与 full prefill 一致性	不同 kernel 可能采用不同 position/cos-sin 路径

高危 Gotcha

不要用 `arange(..., dtype=bfloat16)` 生成 `position`，再乘 `inv_freq`。BF16 尾数精度很低，大位置会出现多个相邻 token 共享同一个浮点位置值的风险。`position index` 应保持整数，`phase` 计算至少在更高精度完成。

17. 2D / 多模态位置：Vision、Image、Video、M-RoPE

文本是 1D 序列，但图像天然 是 2D，视频是 3D（time, height, width）。将视觉 patch 简单 flatten 成 1D token 会让二维邻接关系依赖 flatten order。Vision Transformer 早期常使用 learned 1D positional embedding，并在分辨率变化时对位置表做插值。更现代多模态模型则把 RoPE 扩展到多轴。

Qwen2-VL 提出 Multimodal Rotary Position Embedding (M-RoPE)，将位置分解为时间、height、width 三个分量；文本输入时三个分量使用相同 position id，使其退化回标准 1D RoPE；图像/视频则用多轴坐标。[21]

模态	位置结构	主要 Gotcha
Text	1D token order	packing / chat turn / padding 的 position continuity
Image	2D (h,w)	resize 后 patch grid 改变；不能只按 flatten index 想象空间距离
Video	3D (t,h,w)	fps / temporal sampling 会改变 “一个 time step” 代表的真实时间
Interleaved multimodal	跨模态 + 各模态内部坐标	视觉 token 与文本 token 的跨模态 position discontinuity

18. Position Encoding 与系统设计：FlashAttention、Sliding Window、Cache

Position Encoding 不决定 attention 的 $O(L^2)$ 复杂度，但它会影响 long-context 系统能否安全使用高效 kernel、KV cache 和局部 attention。核心原则是：优化 kernel 不能改变位置语义。

系统组件	与 Position 的耦合
FlashAttention	通常把 RoPE 在 Q/K 生成后或 fused kernel 中应用；必须验证 fused/unfused numerically equivalent
KV Cache	K 一旦按 position 旋转并缓存，decode 时不可再按新 offset 二次旋转
Paged KV Cache	逻辑 token position 与物理 cache page 不同；position 必须跟逻辑 sequence
Sliding Window Attention	可见 edge 被截断；即使 RoPE 可表示长距离，也根本没有 attention edge
Chunked Prefill	每 chunk 的 local index 需要加 global offset
Sequence Packing	若多个独立 sample 共用一个 packed sequence，要决定 position 是否 reset，并保证 attention mask 同步
Context Parallelism	跨 rank 的 token shard 要有全局 position id，不能每卡从 0 开始

19. Gotchas : 35+ 个高频坑

ID	Gotcha	为什么危险
G01	把 max_position_embeddings 改大就当作扩窗	只是 shape/config 变化, 权重没适应新位置。
G02	数学可计算的 PE = 模型会外推	Sinusoidal/RoPE 的函数可计算不代表训练外性能可靠。
G03	把 RoPE 当 additive embedding	RoPE 作用于 Q/K rotation; 错误放在 X 上改变算法。
G04	Q 与 K 使用不同 position offset	相对位移语义直接错误。
G05	K cache 二次旋转	decode 时相位累积。
G06	chunked prefill 每 chunk position 从 0 重启	跨 chunk relation 错乱。
G07	left padding 但 position_ids 仍按 padded index	同一句话因 batch padding 改变绝对/相对坐标。
G08	packed samples 位置 reset 但 mask 不隔离	不同样本会互相看到相同位置空间。
G09	mask 隔离但 position continuity 与训练 recipe 不一致	训练/推理分布 mismatch。
G10	直接修改 rope_theta	所有 position 的相位映射改变, 需要验证 checkpoint 适配。
G11	复制别的模型的 YaRN/LongRoPE 参数	scaling 与原 rope_theta、original_max、head_dim、训练窗口绑定。
G12	忽略 original_max_position_embeddings	很多 scaling 公式需要知道原始训练窗口。
G13	只测试短 prompt	position bug 常在 cache offset 或 >训练长度才暴露。
G14	只做 NIAH	可能高估有效上下文。
G15	只看 perplexity	低 PPL 不等于长距离 retrieval/multi-hop 能力。
G16	只看长任务, 忽略 short retention	context extension 可能牺牲原任务。
G17	BF16 生成 position / phase	长位置精度不足。
G18	cos/sin cache 长度不足后静默 clamp/reuse	高位置复用错误相位。
G19	cos/sin cache 在 device/dtype 切换时错误复用	数值或设备异常。
G20	head_dim 为奇数仍按 pair rotation	RoPE 要按 2D pair 处理, rotary_dim 通常为偶数。
G21	partial rotary 比例与 checkpoint 不匹配	权重预期的几何分解被改变。
G22	GQA 重复 KV 头的顺序错	Q head 对应错误的 K/V group。
G23	跨 batch sequence cache_position 混用	不同序列 decode step 不同, 不能用单一 scalar position。
G24	beam search reorder cache 但 position state 不 reorder	beam 与 KV/position 对不上。
G25	speculative decoding rollback 不回滚 cache position	接受 token 数变化后相位错位。
G26	sliding window 模型只改 RoPE 就宣称全局长上下文	attention edge 仍被窗口截断。
G27	训练只用固定长度 packed 2048	模型可能学到人为边界/绝对长度 prior。
G28	扩窗数据只做重复填充或 random tokens	模型没学真实长程 dependency。
G29	长样本太少	大部分优化 step 没有长距离监督, PE rescale 也难

		被权重适配。
G30	所有层使用相同 scaling 是必然正确	现代架构可能 full/sliding layer 混合，需要 per-layer-type 配置。
G31	视觉 token 只用 1D flatten position	会损失空间邻接 inductive bias。
G32	视频 position 只按 frame index，不考虑 fps	真实时间尺度在不同视频间不一致。
G33	分布式 context parallel 每 rank position 从 0 开始	全局位置被重复。
G34	fused RoPE kernel 与 reference 实现无一致性测试	极端长度/非 contiguous tensor 可能出现差异。
G35	把 “128K config” 当 “128K production guarantee”	必须用 RULER/真实任务、不同位置与不同长度测有效窗口。
G36	修改 tokenizer 后不重新评估 position behavior	同样字符长度变成不同 token 距离，长程依赖尺度也改变。
G37	只讨论 PE，不讨论训练 length curriculum	位置系统提供地址，能力仍需要数据教会模型使用远距离地址。

20. Know-how：训练、扩窗与迁移操作手册

20.1 从零预训练

1. 先确定目标 “有效上下文” 而非只定 max length。把最终业务任务分为 retrieval、multi-hop、aggregation、code/repo、long conversation。
2. 至少做 PE ablation：RoPE vs relative bias/ALiBi（小模型 proxy 即可），同时控制参数量、token budget 和训练 length distribution。
3. 训练长度不要只有一个固定值。混合短/中/长序列，避免模型把 “接近窗口尾部” 当成特殊语义。
4. 记录真实的 pairwise distance histogram，而不仅是 sequence length histogram。两条 64K 样本可能仍只含局部依赖。
5. RoPE 的 theta、rotary_dim、scaling recipe 写进 checkpoint manifest，并在训练与推理加载时做 assert。
6. 长上下文 phase/cos/sin 计算做 FP32 reference test；高效 kernel 结果与 reference 在多长度上比较。

20.2 给已有 RoPE 模型扩窗

7. 冻结基线：先测原模型 1K/2K/4K/原 max 的 PPL、RULER、short benchmarks，作为 retention baseline。
8. 读取原 checkpoint 的 rope_theta、original max、rotary factor、sliding-window 配置；不要从模型名猜。
9. 先做 training-free scaling sanity (PI/dynamic/YaRN 等) 比较，确定哪个方法不会立即破坏短文本。
10. 构造长上下文训练数据：必须包含需要跨越目标距离的信息依赖，而不是单纯把短文 concat。
11. 采用 mixed-window training：短样本保护原能力，长样本教权重使用新的位置区域。
12. 每隔若干 step 同时监控 short PPL 与 long retrieval/multi-hop，避免只优化一个端点。
13. 扩到极长长度时采用 staged/progressive extension 往往比一步跳到目标长度更稳定。
14. 完成后按不同 needle position、不同干扰数量和真实任务测 effective context，而不是只修改 config。

20.3 推理部署

15. 同一模型的训练框架、Transformers、vLLM/SGLang/TensorRT-LLM 等推理栈必须对 RoPE config 语义一致；升级库后做长序列 golden test。
16. 建立 prefill-vs-chunked-prefill、prefill+decode-vs-full-forward 的 logit parity test。
17. 对于 paged KV cache，测试跨 page boundary、cache eviction、beam reorder、spec decode rollback。

18. 对 32K/64K/128K 级 prompt 监控首 token latency、KV memory 与 accuracy；能跑通不是唯一 SLO。

21. Know-why：核心设计背后的因果逻辑

问题	Know-why
为什么要位置编码？	Attention 内容相似度本身没有序列坐标；必须破坏 permutation symmetry。
为什么 relative PE 常比 absolute 更自然？	Attention 的决策是 query-key pairwise relation，距离 i-j 直接对应关系。
为什么 T5 要分桶？	精确区分每个超远距离成本高且未必有用；近距离需要高 resolution，远距离可以 coarse。
为什么 ALiBi 能外推？	距离惩罚函数对任意更大 distance 都有定义，并带有单调 recency inductive bias。
为什么 RoPE 用成对维度？	二维旋转是最小保持范数且能用相位表达位置的正交变换。
为什么 RoPE 内积得到相对位置？	$R(m)^T R(n) = R(n-m)$ ，绝对旋转在点积中相消为相对旋转。
为什么扩窗要缩放频率/位置？	训练外 position 产生 OOD phase；缩放将新距离映射到模型熟悉的相位尺度。
为什么 uniform PI 会伤局部精度？	所有距离除以 s，原本相差 1 的位置在 phase 上变得更接近。
为什么非均匀 scaling 有潜力？	不同频率负责不同尺度，不必为了远距离同时牺牲所有高频 resolution。
为什么 mixed-window training 重要？	只用长样本会让模型适配新位置但忘记原 geometry；短+长共同约束两端。
为什么长上下文还会 Lost-in-the-Middle？	PE 只是地址；attention/训练分布/optimization 仍可能形成首尾偏置和检索弱点。
为什么 tokenizer 会影响位置问题？	PE 的单位是 token，不是字符/单词。同一文本被切得更碎，语义距离对应更大的 token distance。

22. Ablation 与评测框架

一个可靠的 Position Encoding 实验应把“位置算法”“训练长度分布”“计算预算”分开控制。建议最少做下列矩阵。

轴	候选值	必须控制
PE 方法	Absolute / T5 bias / ALiBi / RoPE	model size、optimizer、tokens
RoPE scaling	default / PI / dynamic / YaRN / candidate	original theta、fine-tune tokens
训练长度	2K / mix 2-8K / mix 2-32K	总 token 数而非 step 数
目标测试长度	1× / 2× / 4× / 8× / 16×	同一任务生成方式
needle 位置	0/25/50/75/100%	needle 内容固定
任务复杂度	single / multi-key / multi-hop / aggregation	context noise distribution
dtype	FP32 reference / BF16 / FP16	相同 kernel
kernel	reference / SDPA / FlashAttention / fused RoPE	相同 logits tolerance

22.1 建议指标

- Perplexity vs context length 曲线，而不是单点。
- Accuracy heatmap: x=needle position, y=context length。
- Short-context retention ratio。
- Long-context effective length: 达到某最低 accuracy 的最大长度。
- Attention entropy / logit statistics（用于发现 scaling 后 attention 过尖/过平）。
- Throughput、prefill latency、KV memory；PE 方案必须放进系统总成本。

23. 方法选择决策树

场景	优先选择	理由
从零训练通用 decoder LLM	RoPE（或成熟 model-family recipe）	生态成熟，兼容现代 attention/KV 工具链
极简、强调 train-short-test-long	ALiBi 可做强 baseline	实现和参数极简，天然 distance bias
Encoder-decoder / T5-like	Relative position bias	与架构成熟匹配，参数少
已有 RoPE checkpoint 扩窗 2-8×	先评估 PI / dynamic / YaRN	改动小，成熟实现多
已有 RoPE checkpoint 扩到极长	LongRoPE/LongRoPE2 类 + staged training	需要非均匀 scaling 与 mixed context
需要语义计数/结构位置	CoPE 类研究方向	token count 未必是最佳地址
图像/视频多模态	2D/3D RoPE 或 model-native M-RoPE	保留空间/时间坐标
只需处理超长知识库	先比较 RAG / sparse attention	扩 PE 不等于必须把所有 token dense attend

推荐原则

优先沿用目标 checkpoint 原生 Position Encoding 与官方扩窗 recipe。Position 机制与权重共同训练后已经形成耦合，后期“换一种更先进 PE”通常不是无损插件。

24. PyTorch From Scratch：关键实现

24.1 Sinusoidal PE

```
import torch

def sinusoidal_positions(seq_len, dim, base=10000.0, device=None):
    pos = torch.arange(seq_len, device=device, dtype=torch.float32)[: , None]
    i = torch.arange(0, dim, 2, device=device, dtype=torch.float32)
    inv_freq = base ** (-i / dim)
    angle = pos * inv_freq[None, :]
    pe = torch.empty(seq_len, dim, device=device, dtype=torch.float32)
    pe[:, 0::2] = angle.sin()
    pe[:, 1::2] = angle.cos()
    return pe
```

24.2 RoPE reference implementation

```
def rotate_half(x):
    # Pair dimensions: (x0,x1),(x2,x3),...
    x_even = x[..., 0::2]
    x_odd = x[..., 1::2]
    y = torch.stack((-x_odd, x_even), dim=-1)
    return y.flatten(-2)

def rope_cos_sin(position_ids, dim, theta=10000.0, device=None):
    Position Embedding 深度分析 2026 · 19
```

```
# Important: position stays integer; phase is formed in FP32.
inv = theta ** (-torch.arange(0, dim, 2, device=device,
                             dtype=torch.float32) / dim)
phase = position_ids.to(torch.float32)[..., None] * inv
phase = torch.repeat_interleave(phase, 2, dim=-1)
return phase.cos(), phase.sin()
```

```
def apply_rope(x, cos, sin):
    # x: [B,H,T,D], cos/sin: [B,T,D] -> unsqueeze head axis
    cos = cos.unsqueeze(1)
    sin = sin.unsqueeze(1)
    return x * cos + rotate_half(x) * sin
```

24.3 Linear Position Interpolation

```
def interpolated_position_ids(seq_len, train_max, target_max, device=None):
    factor = target_max / train_max
    p = torch.arange(seq_len, device=device, dtype=torch.float32)
    return p / factor # feed this effective coordinate into RoPE phase
```

24.4 必须写的 parity test

```
# Test A: full prefill vs chunked prefill logits
# Test B: full forward last-token logit vs prefill + decode
# Test C: reference FP32 RoPE vs fused kernel at 2K/8K/32K/128K
# Test D: same unpadded text alone vs left/right-padded batch
# Test E: KV cache reorder / beam reorder / speculative rollback
```

25. 深度面试题 / Debug 题

1. 为什么没有位置编码的 self-attention 仍然不是完全 permutation-invariant，而是 permutation-equivariant？两者有什么区别？
2. 推导 sinusoidal PE 中 $PE(pos+k)$ 为什么可以由 $PE(pos)$ 的线性变换表达。
3. Learned absolute embedding 为什么在训练长度内可能比 sinusoidal 更灵活，但外推更差？
4. 解释 Shaw relative position 与 T5 relative bias 的区别：一个是向量，一个是标量，它们分别进入 attention 的哪里？
5. 为什么 T5 采用 logarithmic buckets，而不是每个 relative distance 一个参数？
6. 写出 ALiBi 的 attention score，并解释不同 head slope 的意义。
7. 从二维旋转矩阵证明 RoPE 的 $q_m^T k_n$ 只依赖 $n-m$ 。
8. RoPE 为什么保持向量 norm？这对 attention 数值有什么意义？
9. rope_theta 增大时，频率和波长如何变化？为什么这可能帮助长上下文但不能直接改 checkpoint？
10. 为什么 Position Interpolation 能比 naive extrapolation 更稳定？它牺牲了什么？
11. YaRN 为什么需要 attention scaling，而不仅是 position scaling？
12. LongRoPE 的 “non-uniformity” 具体指哪些维度？
13. 一个模型 config 声称 128K，如何设计实验判断 “有效 128K”？
14. 为什么 Needle-in-a-Haystack 不足以证明 long-context reasoning？
15. 解释 Lost-in-the-Middle 与 Position Encoding 的关系：为什么不能简单说 “都是 RoPE 的错”？
16. BF16 为什么可能破坏长上下文 RoPE？position_ids 本身应该使用什么 dtype？
17. 在 GQA 中 RoPE 应在 KV repeat 前还是后做？两种都能跑时，哪个更自然，为什么？
18. chunked prefill 时每个 chunk 从 position 0 开始会发生什么？如何写单元测试抓住它？
19. PagedAttention 的 cache block id 为什么不能直接当 position id？

20. 如果更换 tokenizer, 为什么模型的有效 position-distance distribution 会改变?
21. 多模态模型为什么需要 2D/3D position, 而不是把 patch/video token 全部 1D 编号?
22. 如果模型同时使用 sliding-window attention 和 RoPE, 扩大 RoPE 窗口是否能获得全局 attention?
23. 设计一个 1B proxy experiment, 比 RoPE、ALiBi 与 T5 bias, 如何控制公平性?
24. 为什么“PE 参数越少越好”不是正确优化目标?
25. 如果 long-context PPL 变好但 multi-hop accuracy 变差, 你会检查哪些因素?

26. 参考文献与研究来源

- [1] Vaswani et al. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.
- [2] Shaw, Uszkoreit, Vaswani. Self-Attention with Relative Position Representations. NAACL 2018. arXiv:1803.02155.
- [3] Dai et al. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. ACL 2019. arXiv:1901.02860.
- [4] Raffel et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 2020. arXiv:1910.10683.
- [5] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. arXiv:1810.04805.
- [6] Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864; Neurocomputing 2024.
- [7] Press, Smith, Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. arXiv:2108.12409.
- [8] Dufter, Schmitt, Schütze. Position Information in Transformers: An Overview. arXiv:2102.11090.
- [9] Sun et al. A Length-Extrapolatable Transformer (xPos). arXiv:2212.10554.
- [10] Chen et al. Extending Context Window of Large Language Models via Positional Interpolation. arXiv:2306.15595.
- [11] Peng et al. YaRN: Efficient Context Window Extension of Large Language Models. arXiv:2309.00071.
- [12] Ding et al. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. arXiv:2402.13753.
- [13] Shang et al. LongRoPE2: Near-Lossless LLM Context Window Scaling. arXiv:2502.20082.
- [14] Grattafiori et al. The Llama 3 Herd of Models. arXiv:2407.21783.
- [15] Hugging Face Transformers. Utilities for Rotary Embedding / RoPE configuration (official documentation, accessed 2026-08).
- [16] Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024 / arXiv:2307.03172.
- [17] Hsieh et al. RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.
- [18] Chiang, Yogatama. The Rotary Position Embedding May Cause Dimension Inefficiency in Attention Heads for Long-Distance Retrieval. arXiv:2502.11276.
- [19] Golovneva et al. Contextual Position Encoding: Learning to Count What's Important. arXiv:2405.18719.
- [20] Wang et al. When Precision Meets Position: BFloat16 Breaks Down RoPE in Long-Context Training. arXiv:2411.13476.
- [21] Wang et al. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. arXiv:2409.12191.
- [22] Zhang et al. Found in the Middle: How Language Models Use Long Contexts Better via Plug-and-Play Positional Encoding. arXiv:2403.04797.
- [23] Men et al. Understanding the RoPE Extensions of Long-Context LLMs. arXiv:2406.13282.
- [24] Wu, Liu, Jadbabaie. How Data Shapes RoPE Frequency Usage: From Positional Scale Matching to Length Generalization. arXiv:2607.07678 (2026 preprint).

[25] Du et al. RoPE Distinguishes Neither Positions Nor Tokens in Long Context. arXiv:2605.15514 (2026 preprint).

证据分级说明

本报告把 [1]-[17]、[19]-[21] 这类已广泛使用/复现的方法作为主线；2025-2026 的频率利用、维度效率和新 failure analysis 作为“新研究/待进一步复现”呈现。某篇论文报告的最长 context 数字不应直接等价于所有模型、任务和硬件下的生产保证。

最终一句话

Position Encoding 的目标不是“让模型知道第几个 token”这么简单，而是构造一个可训练、可比较、可外推、数值稳定且能与 attention 系统共同工作的地址几何。现代长上下文优化，本质上是在重新设计这个地址几何与训练数据之间的匹配关系。